

AI FITTRACK

PERSONALIZED FITNESS RECOMMENDATIONS POWERED BY AI

Project Documentation

TEAM MEMBERS

Table 2: Team Members and Responsibility

No.	Name	Responsibility
1	CHANDRA SEKAR G	Team Leader — Project integration, backend coordination, AI integration, final testing and debugging, GitHub management, demo coordination and Skill Wallet submission
2	CHANDRU G T	Backend Development and API Integration — route configuration, controller implementation, middleware integration, API functionality and debugging support
3	CHANDRU M	Database Configuration and Design — MongoDB Atlas configuration, User and Workout schemas, relationships, validation and database verification
4	BOOPATHI B L	API Testing and Verification — authentication APIs, workout CRUD and search operations, AI endpoints, test evidence and regression testing
5	CHANDAN KUMAR B	Architecture, Documentation and Execution Evidence — architecture diagrams, project structure documentation, screenshots, README preparation and repository organization

1. INTRODUCTION

1.1 Project Background

Fitness applications require effective mechanisms for recording exercise activities, managing workout history and presenting meaningful information to users. When workout records are maintained manually, organizing historical activities, searching previous sessions and relating workout activity to personal fitness goals can become difficult. A backend service can centralize these records and provide consistent programmatic access to them.

AI FitTrack addresses this requirement through a RESTful backend rather than a standalone user-interface application. The supplied project documentation identifies secure authentication, workout management, search, AI recommendation and fitness insight generation as the principal functions. The architecture is modular so that database access, authentication, route handling and AI processing remain separated.

1.2 Problem Statement

The project addresses the challenges of maintaining a centralized workout history while protecting personal fitness information. Manual records can be difficult to search and do not inherently provide personalized fitness guidance. The system therefore requires authenticated access, structured workout records and endpoints that can retrieve information according to the authenticated user.

A second requirement is intelligent assistance. A user may provide basic information about age, fitness goal and experience level and request a workout recommendation. Similarly, workout statistics can be submitted to obtain an AI-generated fitness insight. The supplied FitTrack specification identifies these two AI functions as core features.

1.3 Aim

The aim of AI FitTrack is to provide a secure backend service that combines authenticated workout management with AI-assisted fitness guidance through RESTful APIs.

The project is designed as an academic backend implementation. It can be consumed by an API client and can conceptually support a web or mobile frontend, but the supplied implementation itself is the backend API and does not contain a separate React application.

1.4 Objectives

- To Implement secure user registration, login and profile retrieval.
- To Protect authenticated routes using JWT middleware.
- To Hash passwords using bcryptjs before storage.
- To Provide complete workout CRUD operations.
- To Provide workout search by name, category and date.
- To Integrate Google Gemini for workout recommendations.
- To Integrate Google Gemini for fitness insights.
- To Maintain a modular MVC-oriented backend structure.
- To Provide evidence suitable for Skill Wallet and GitHub review.

2. SYSTEM STUDY

2.1 Existing System

In the existing approach described by the assigned case study, workout information may be maintained manually using notebooks or spreadsheets. Such a process can store information but does not inherently provide authenticated API access, structured search, or an integrated AI assistance mechanism.

Manual tracking also places responsibility on the user to calculate or interpret workout statistics. A backend service can instead standardize data entry, validation, retrieval and ownership checks and can provide an API boundary through which intelligent services can be invoked.

2.2 Drawbacks of the Existing Approach

- Workout history can be difficult to organize and search.
- Personal fitness records may lack application-level authentication.
- Repeated manual entry and calculations can be time-consuming.
- Personalized workout recommendations are not automatically available.
- Different records may not follow a consistent data structure.

The assigned FitTrack case study identifies centralized-record, authentication, search and personalized-guidance problems as the motivation for the proposed backend.

2.3 Proposed System

AI FitTrack provides a centralized backend through which authenticated users can register, log in and manage their workout records. JWT middleware protects the secured endpoints and the Workout model associates each record with the authenticated user's ObjectId. REST endpoints provide create, retrieve, search, update and delete operations.

The proposed system also provides two AI functions. The recommendation endpoint receives age, fitness goal and experience level. The fitness-insights endpoint receives total workouts, average duration and total calories burned. Google Gemini is accessed through a dedicated service module and the resulting text is returned to the API consumer.

2.4 Key benefits

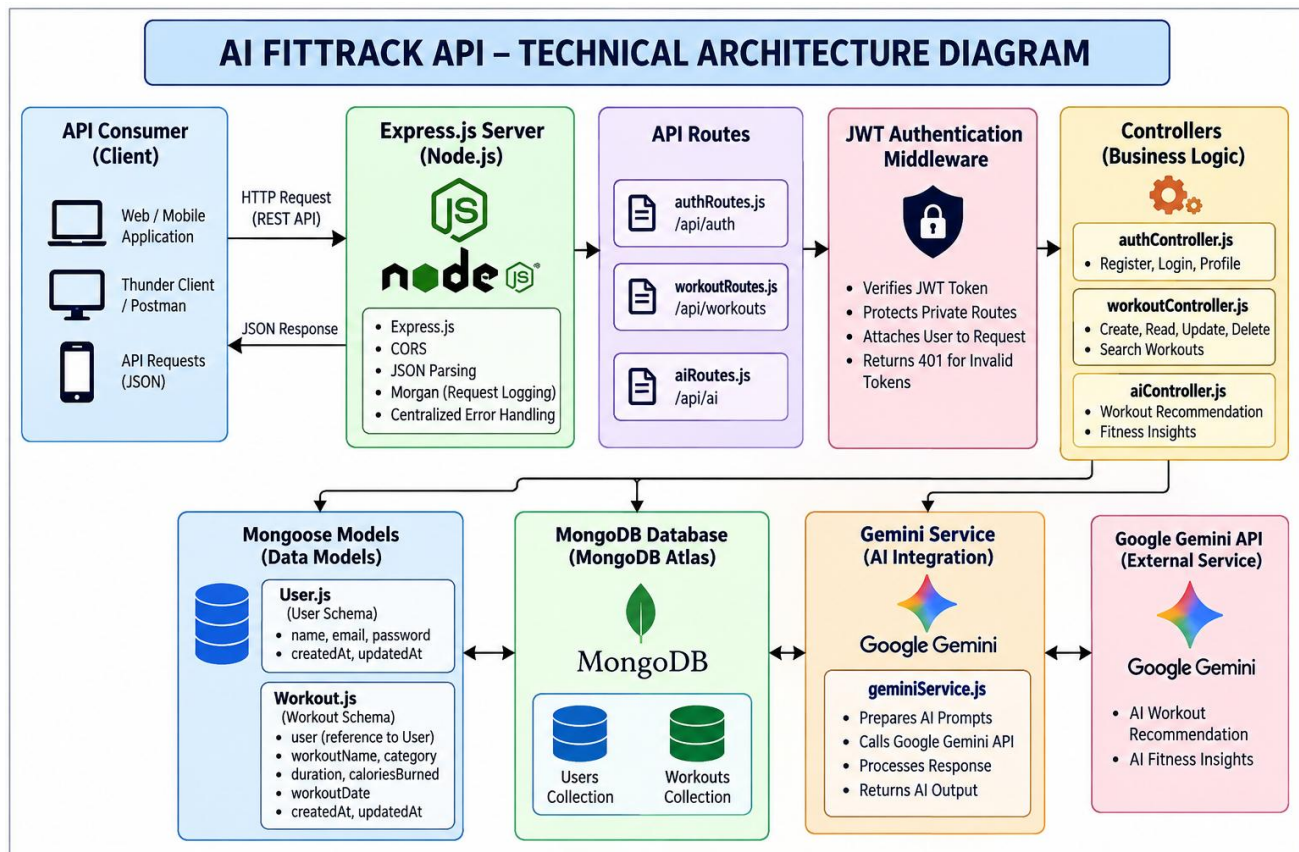
- Centralized workout storage.
- JWT-protected access to user-specific data.
- Password hashing using bcryptjs.
- CRUD and search operations through REST endpoints.
- AI-assisted recommendations and insights.
- Modular code organization.
- JSON-based responses suitable for API consumers.

3. SYSTEM DESIGN AND DEVELOPMENT

3.1 Technical Architecture

The AI FitTrack API follows a layered architecture based on MVC principles. A request enters through the Express application, is routed to an endpoint, passes through JWT protection where required, and is processed by a controller. Controllers communicate with Mongoose models for persistent data operations or invoke the Gemini service for AI functionality.

The architecture separates routing, authentication, business processing, data access and AI integration. The supplied reference identifies the major components as the client/API consumer, Express server, API routes, authentication middleware, controllers, services, Google Gemini AI service, Mongoose models and MongoDB database.



3.2 Component Design

Table 3: Major Architecture Components

Component	Responsibility
Client / API Consumer	Sends HTTP requests and receives JSON responses.
Express Server	Starts the server, parses JSON, enables CORS, registers routes and handles errors.
Authentication Middleware	Extracts and verifies JWT and attaches the authenticated user.
Routes	Maps HTTP endpoints to their corresponding controller functions.
Controllers	Validates input, executes application logic and generates structured responses.
Models	Define Mongoose schemas and interact with MongoDB.
Gemini Service	Prepares prompts and communicates with the Google Gemini service.
MongoDB	Stores User and Workout documents.

AI FITTRACK - KEY FEATURES

SECURE USER AUTHENTICATION

- User registration
- User login
- JWT-based authentication
- Protected user profile access
- Password hashing using bcrypt

WORKOUT MANAGEMENT

- Add workout records
- View all workouts
- View workout by ID
- Update workout details
- Delete workout records

WORKOUT SEARCH

- Search by workout name
- Search by category
- Search by workout date

AI FEATURES

- AI workout recommendation
- AI fitness insights
- Google Gemini integration

MONGODB DATABASE

- MongoDB Atlas
- Mongoose data modeling
- Users collection
- Workouts collection

RESTFUL API

- Express.js REST API
- GET, POST, PUT and DELETE operations
- JSON request and response format

MIDDLEWARE & ERROR HANDLING

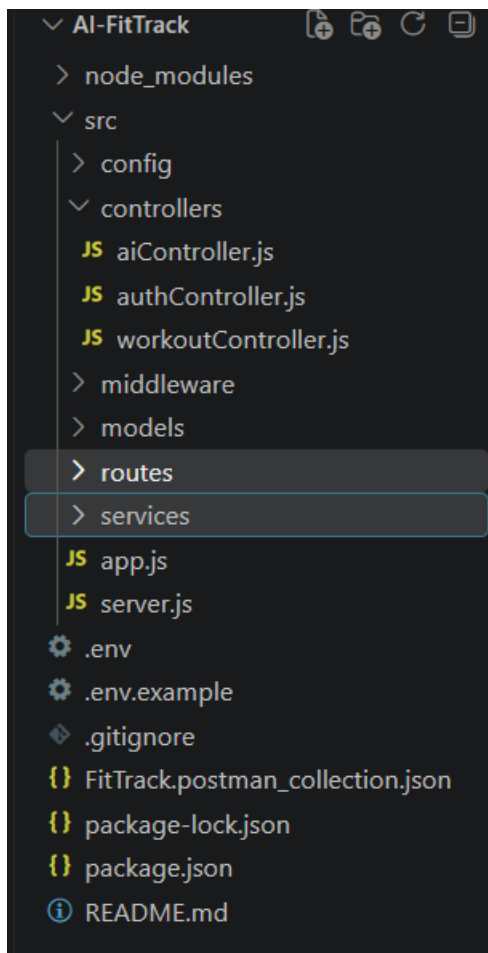
- JWT authentication middleware
- CORS support
- Morgan request logging
- Centralized error handling

MODULAR MVC ARCHITECTURE

- Models for database structures
- Controllers for business logic
- Routes for API endpoints
- Separate Gemini service for AI integration

Skill Wallet - Task 3: Features | AI FitTrack

3.3 File Design



The actual supplied source separates the application into configuration, controllers, middleware, models, routes and services. This structure is retained in the GitHub documentation rather than replacing it with a different architecture.

3.4 Input Design

Table 4: Major API Inputs

Function	Required Input
Register	name, email, password
Login	email, password
Add Workout	workoutName, category, duration, caloriesBurned, workoutDate
Search	q query parameter
Workout Recommendation	age, fitnessGoal, experience
AI Fitness Insights	totalWorkouts, averageDuration, totalCaloriesBurned

3.5 Output Design

The backend returns JSON responses with status codes that indicate the outcome of each request. Authentication responses contain a JWT token and basic user information, while workout responses contain the relevant workout record or collection data. AI responses contain generated workout recommendations or fitness insights.

The root endpoint also provides a health response that confirms the API is running. This endpoint was verified during execution on localhost:5000.

3.6 Database Design

MongoDB stores documents in collections, while Mongoose provides schemas, validation and application-level database methods. The detailed FitTrack reference explicitly identifies Users and Workouts as the two collections in the implemented database design.

3.6.1 User Entity

Table 5: User Entity

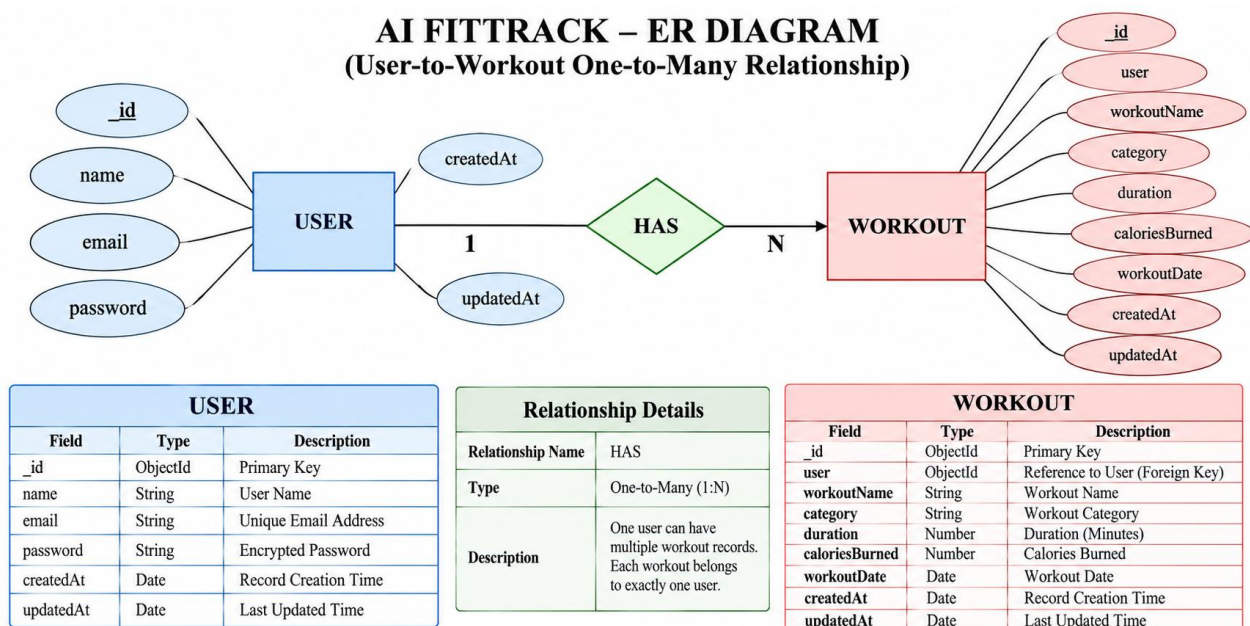
Field	Type	Description
_id	ObjectId	Primary key
name	String	Required user name
email	String	Unique, lowercased email
password	String	Hashed password
createdAt	Date	Automatic timestamp
updatedAt	Date	Automatic timestamp

3.6.2 Workout Entity

Table 6: Workout Entity

Field	Type	Description
_id	ObjectId	Primary key
user	ObjectId	Reference to User
workoutName	String	Required workout name
category	String	Supported workout category
duration	Number	Minutes; minimum 1
caloriesBurned	Number	Minimum 0
workoutDate	Date	Workout date
createdAt / updatedAt	Date	Automatic timestamps

A User can create multiple Workout records, while each Workout belongs to one authenticated user. The relationship is maintained by the user ObjectId stored in the Workout document.



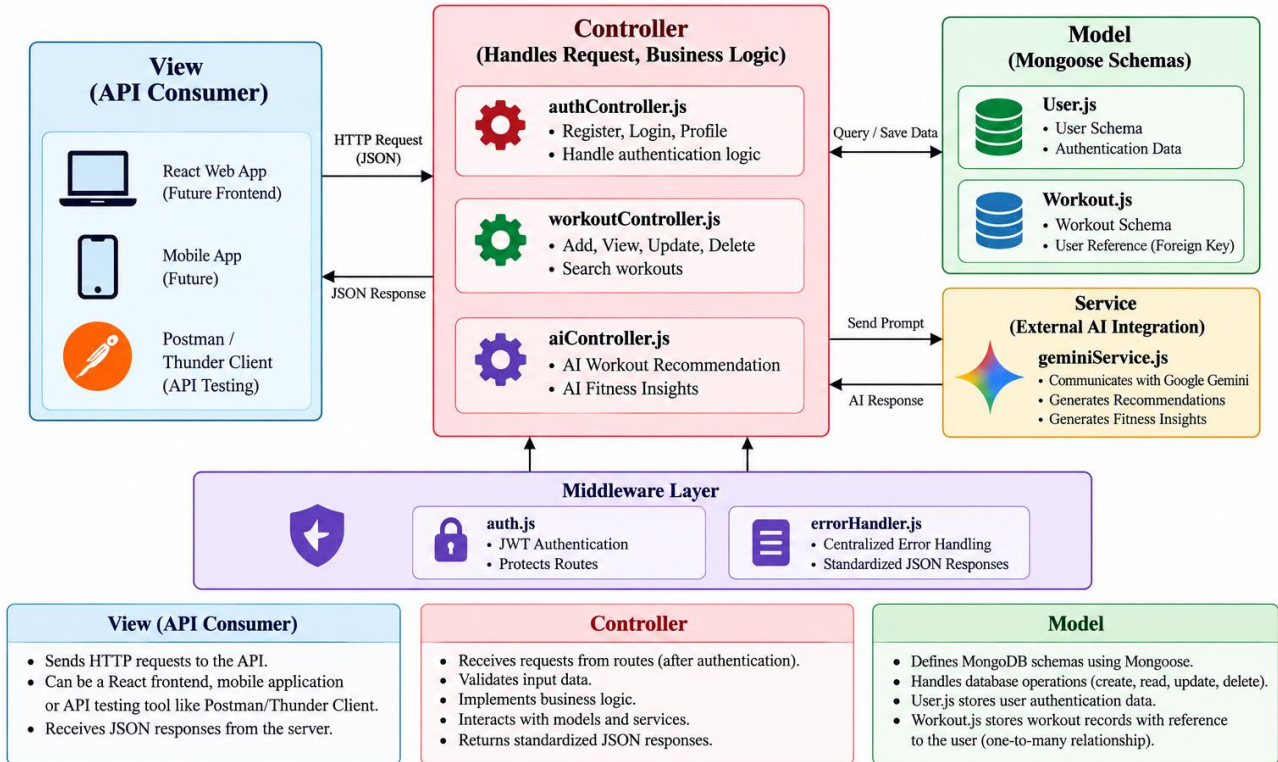
3.7 MVC Pattern

The Model layer is implemented through User.js and Workout.js. These files define Mongoose schemas, validation rules and database mappings. The Controller layer contains authController.js, workoutController.js and aiController.js, which process requests and coordinate data or AI operations.

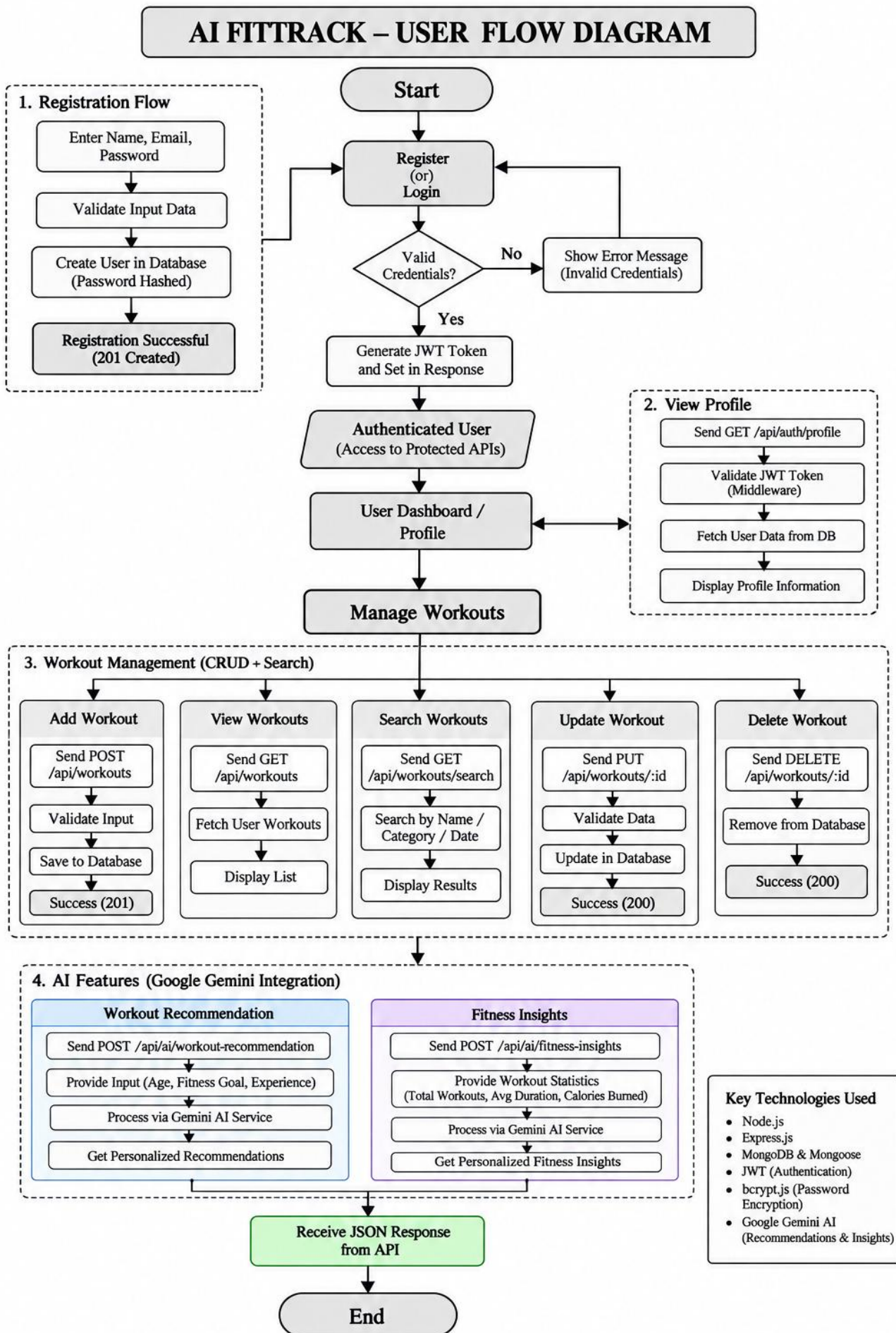
Because the supplied project is a REST API, the View layer is represented by API consumers such as Thunder Client or Postman. It can also be consumed conceptually by web or mobile clients.

AI FITTRACK – MVC ARCHITECTURE DIAGRAM

(Model – View – Controller)



3.8 USER FLOW



4. TESTING AND IMPLEMENTATION

4.1 Project Setup

The project was executed on Windows using Node.js and npm. Dependencies were installed using npm install. MongoDB Atlas was configured after the initial local MongoDB connection attempt returned an ECONNREFUSED error. The Atlas connection was then configured through MONGO_URI and the application successfully connected.

The application was started with npm run dev and ran on port 5000. The root endpoint returned the success message 'AI FitTrack API is running successfully' with version 1.0.0. The server terminal also displayed a successful MongoDB connection.

The image shows two screenshots. The top screenshot is a terminal window with the following output:

```
PS D:\projects\AI-FitTrack> npm run dev
npm notice run ai-fittrack-api@1.0.0 dev
npm notice run nodemon src/server.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
Server running in development mode on port 5000
MongoDB Connected: ac-srthtjc-shard-00-00.lh6ijvv.mongodb.net
```

The bottom screenshot is a web browser window showing the response from the root endpoint (localhost:5000). The response is a JSON object: {"success":true,"message":"AI FitTrack API is running successfully","version":"1.0.0"}.

4.2 Authentication API Testing

Table 7: Authentication Test Results

Test	Endpoint	Observed Result
Register	POST /api/auth/register	201 Created — user and JWT returned
Login	POST /api/auth/login	200 OK — user and JWT returned
Profile	GET /api/auth/profile	200 OK — authenticated profile returned

The registration test created a test account. The login request returned a JWT, which was subsequently supplied through the Bearer Authorization header when testing the profile endpoint.

Register User

The screenshot shows the Thunder Client interface with a new request configured. The request is a POST to `http://localhost:5000/api/auth/register`. The body is a JSON object with the following content:

```
1 {
2   "name": "FitTrack Test User",
3   "email": "fittrack.test@example.com",
4   "password": "Test@12345"
5 }
```

The response status is **201 Created**, with a size of **303 Bytes** and a time of **608 ms**. The response body is a JSON object:

```
1 {
2   "success": true,
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZlZHVjU10thkMTkxYzBlZDA2NmVhMzhkZSIsIm1hdCI6MTc5MDI2OTgzNjYwIiwiaXwiOiJoxNzkyODYxODM3fQ.CE89IHaqTUP7C2M_m03UUK8B8Dnu9T3nb6J3eMeMBU1I",
4   "user": {
5     "id": "6ab5598d131c0ed035ea38de",
6     "name": "FitTrack Test User",
7     "email": "fittrack.test@example.com"
8   }
9 }
```

Login User

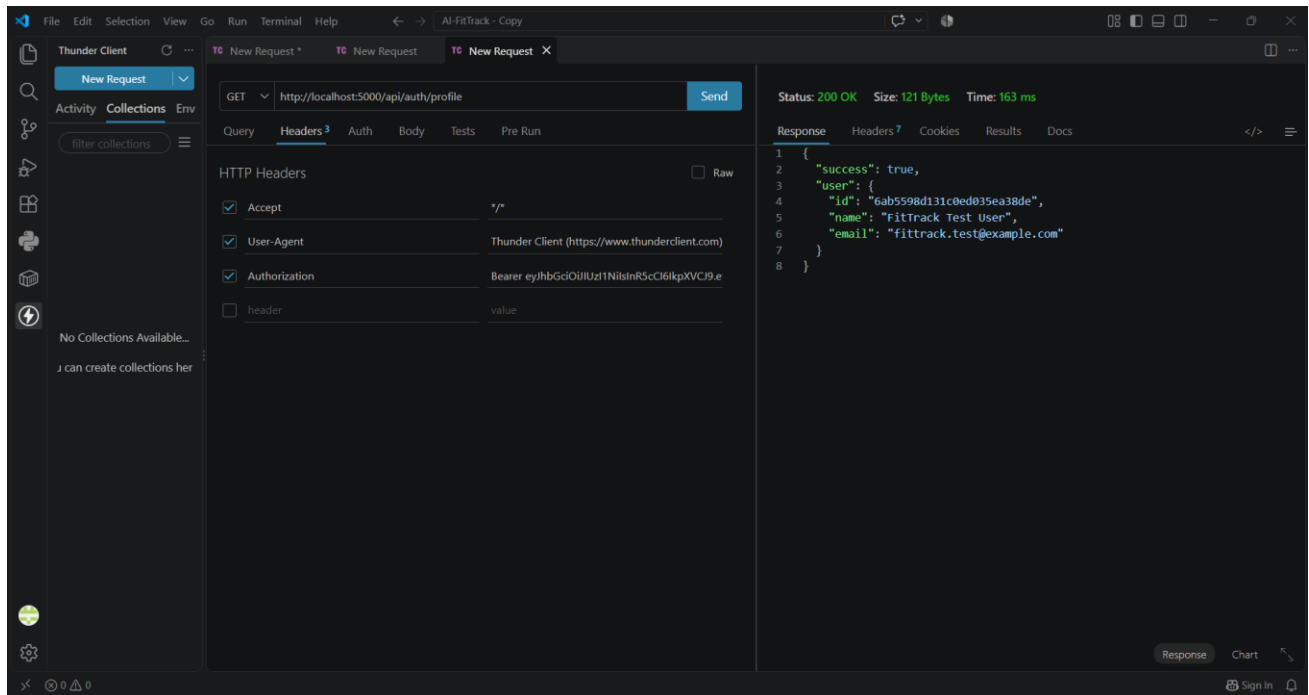
The screenshot shows the Thunder Client interface with a new request configured. The request is a POST to `http://localhost:5000/api/auth/login`. The body is a JSON object with the following content:

```
1 {
2   "email": "fittrack.test@example.com",
3   "password": "Test@12345"
4 }
```

The response status is **200 OK**, with a size of **303 Bytes** and a time of **346 ms**. The response body is a JSON object:

```
1 {
2   "success": true,
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZlZHVjU10thkMTkxYzBlZDA2NmVhMzhkZSIsIm1hdCI6MTc5MDI2OTgzNjYwIiwiaXwiOiJoxNzkyODYxODM3fQ.CsREi6LPdorz2VXzHctzSZzcqEh5haz7G91yE2BK9FE",
4   "user": {
5     "id": "6ab5598d131c0ed035ea38de",
6     "name": "FitTrack Test User",
7     "email": "fittrack.test@example.com"
8   }
9 }
```

Get Profile



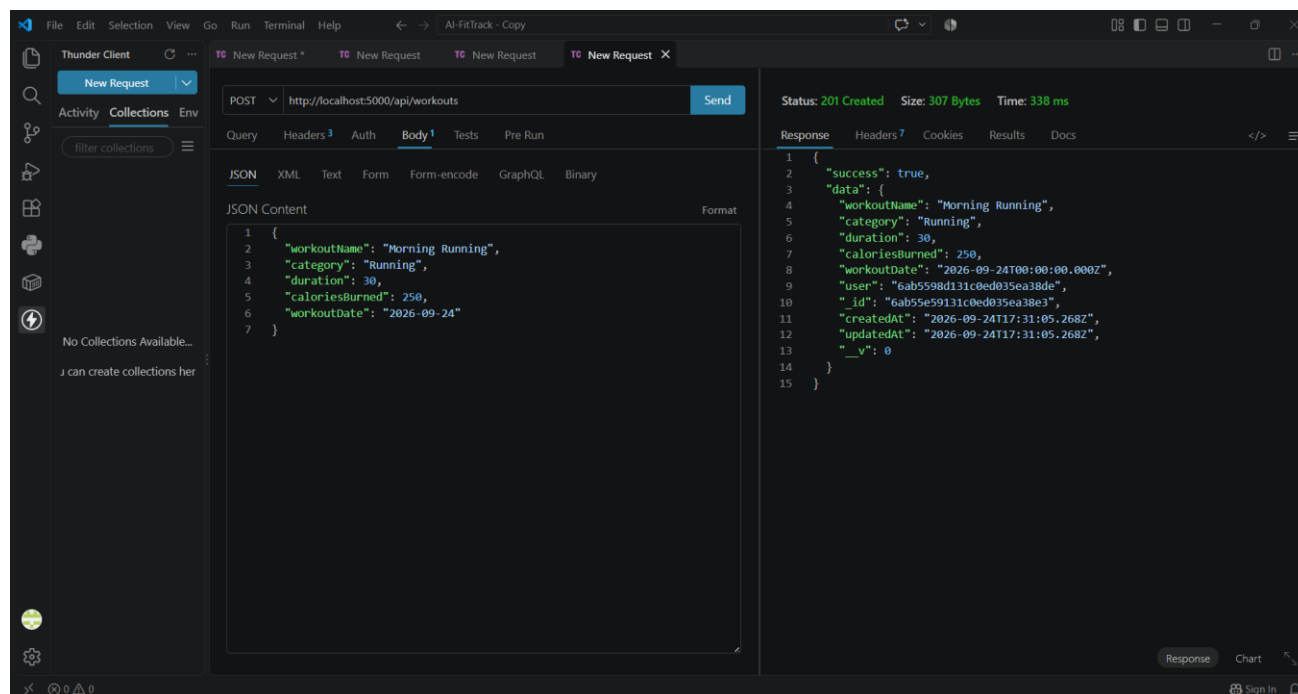
4.3 Workout API Testing

Table 8: Workout API Test Results

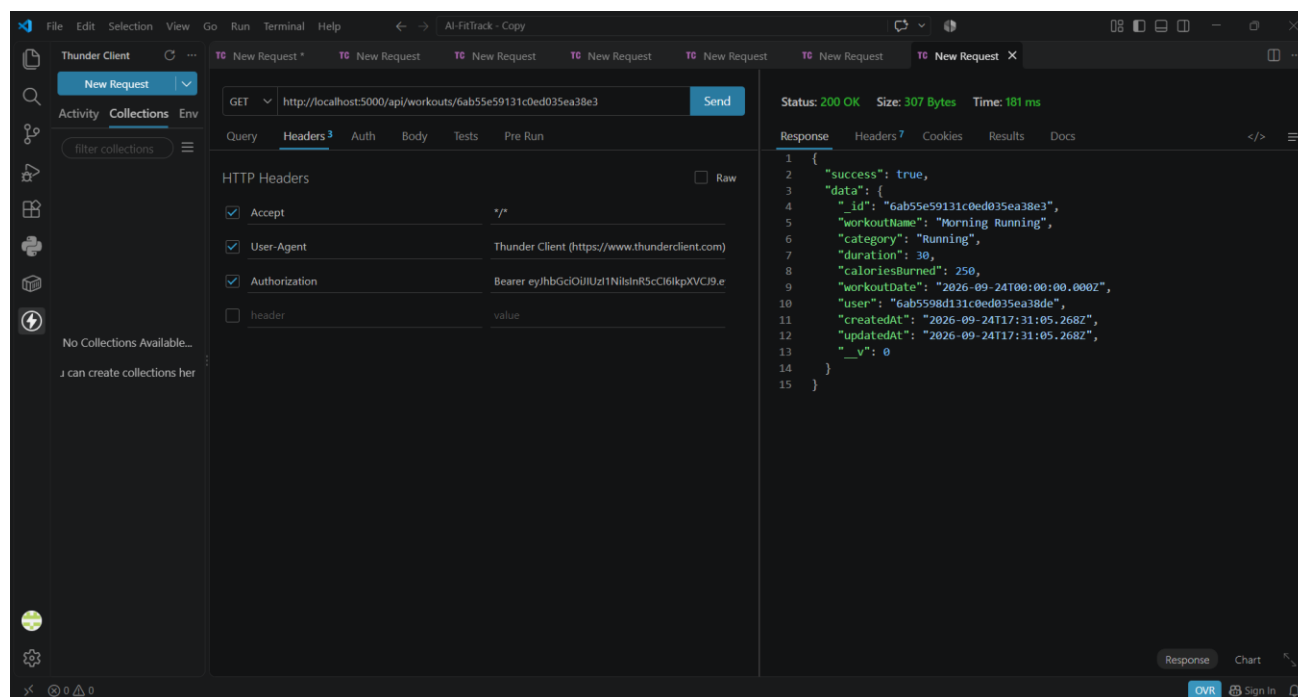
Operation	Endpoint	Observed Result
Create	POST /api/workouts	201 Created
Get All	GET /api/workouts	200 OK
Search Name	GET /api/workouts/search?q=Morning	200 OK
Search Category	GET /api/workouts/search?q=Running	200 OK
Search Date	GET /api/workouts/search?q=2026-09-24	200 OK
Get by ID	GET /api/workouts/:id	200 OK after exact ID was used
Update	PUT /api/workouts/:id	200 OK
Delete	DELETE /api/workouts/:id	200 OK
Post-delete Get All	GET /api/workouts	200 OK; empty result

The complete workout CRUD cycle was successfully tested using a test workout record. Workout search was verified using workout name, category and date-based queries. During testing, an initial Get-by-ID request used an incorrect identifier and returned HTTP 404. The request was subsequently repeated using the correct generated identifier and returned HTTP 200 OK. This is retained as a testing observation rather than hidden.

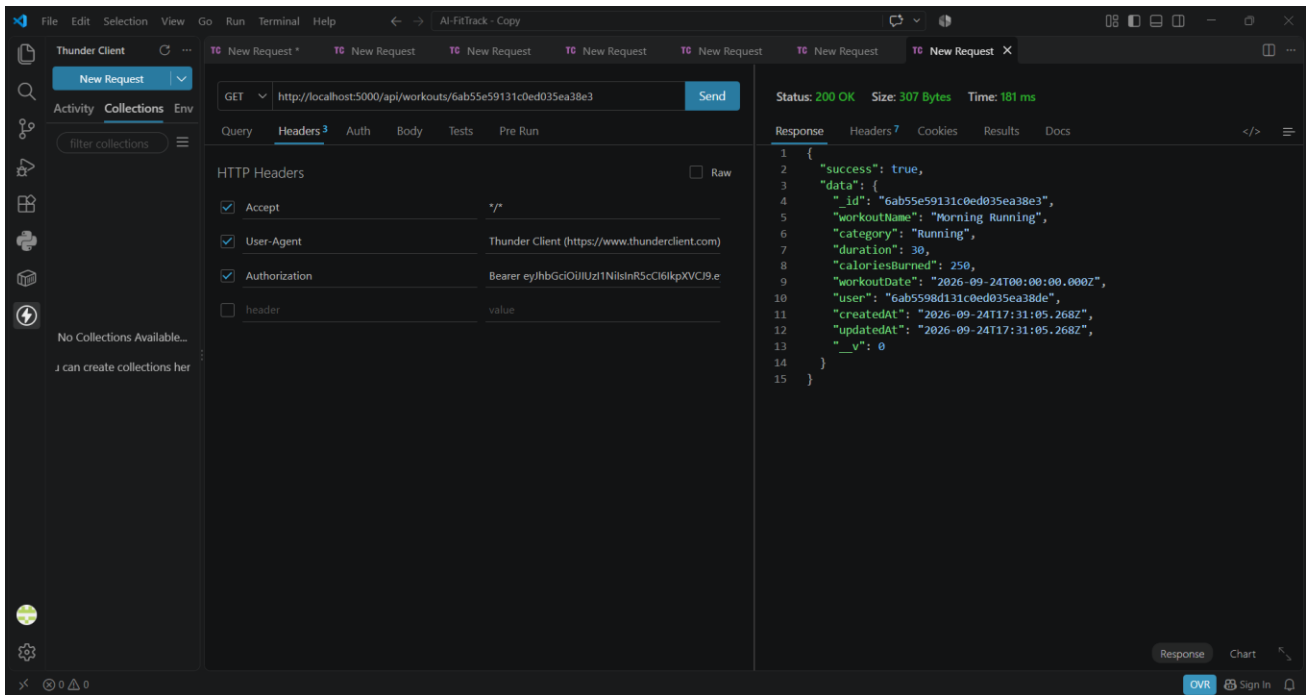
Add Workout



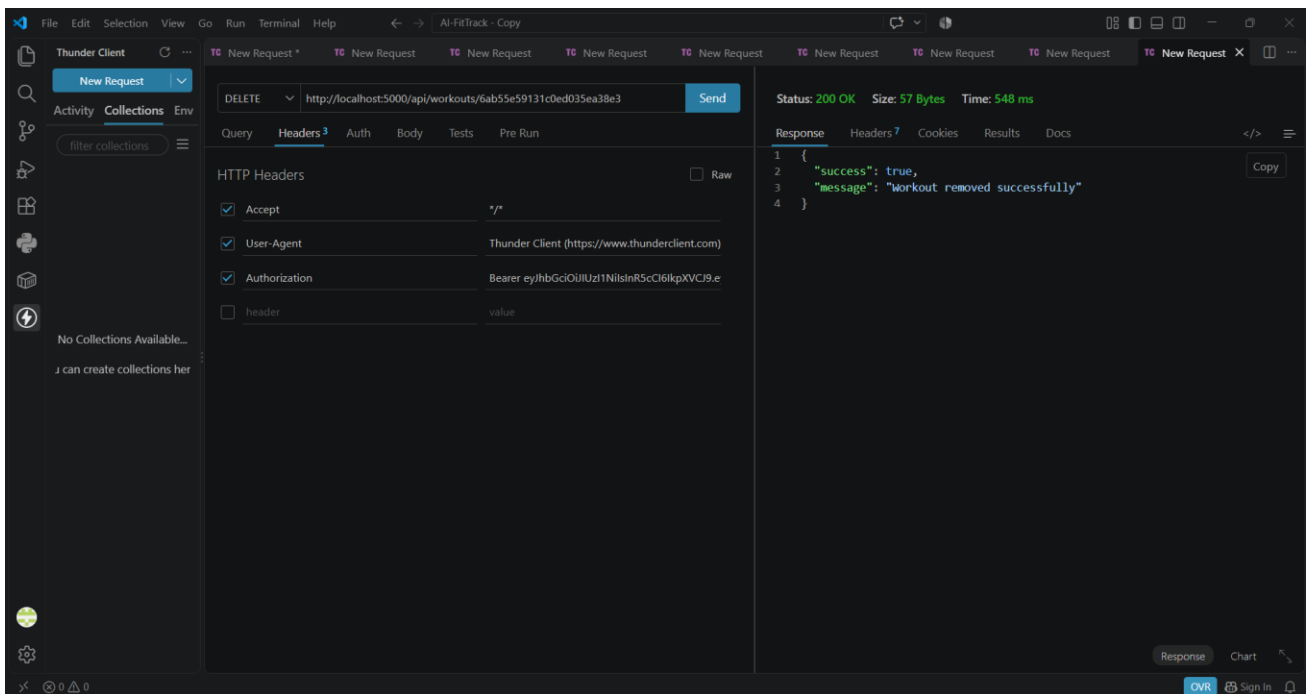
Get Workout By ID



Update Workout



Delete Workout

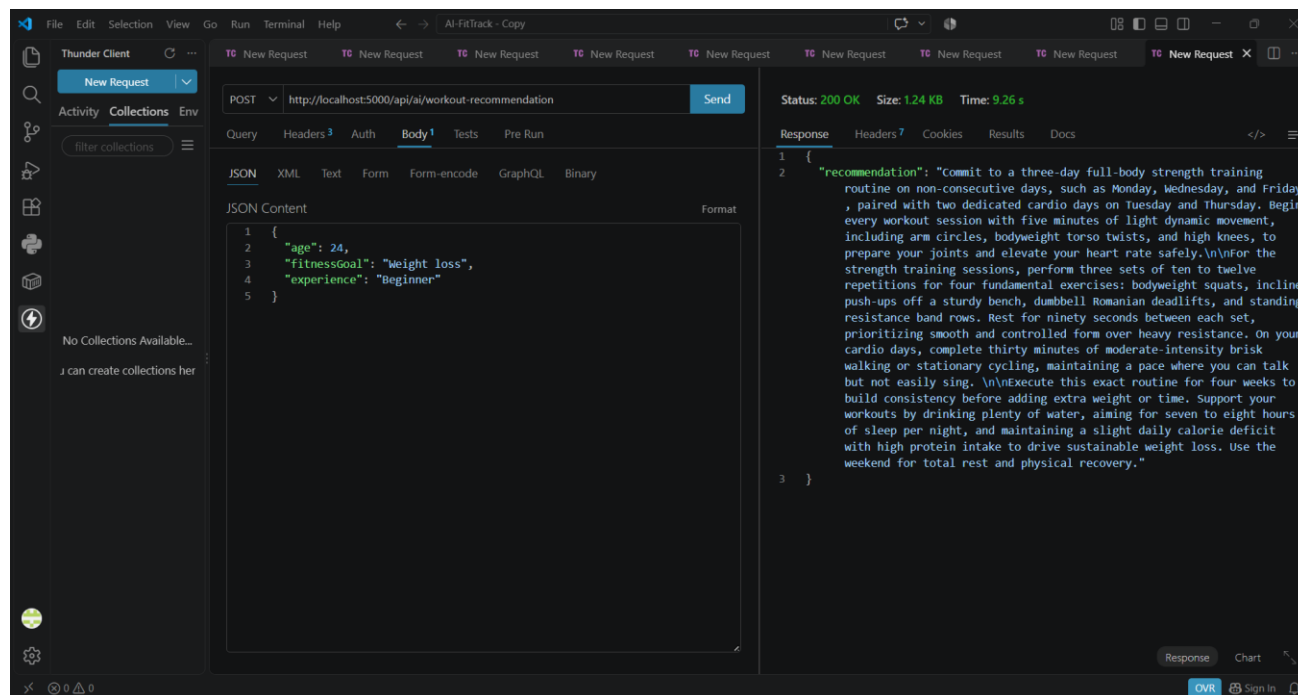


4.4 AI API Testing

The first workout-recommendation request returned HTTP 500 because the configured Gemini model was no longer available to new users. The terminal error identified the model as unavailable and directed the project to a current model. The planned correction is to set `GEMINI_MODEL` to the current supported model and to make both AI functions use the environment-configured model.

At the point of document preparation, the final post-correction 200 OK evidence for both AI endpoints had not yet been independently confirmed in the conversation record. Therefore this document intentionally marks the final AI evidence as pending rather than claiming a successful result without evidence.

Workout Recommendation



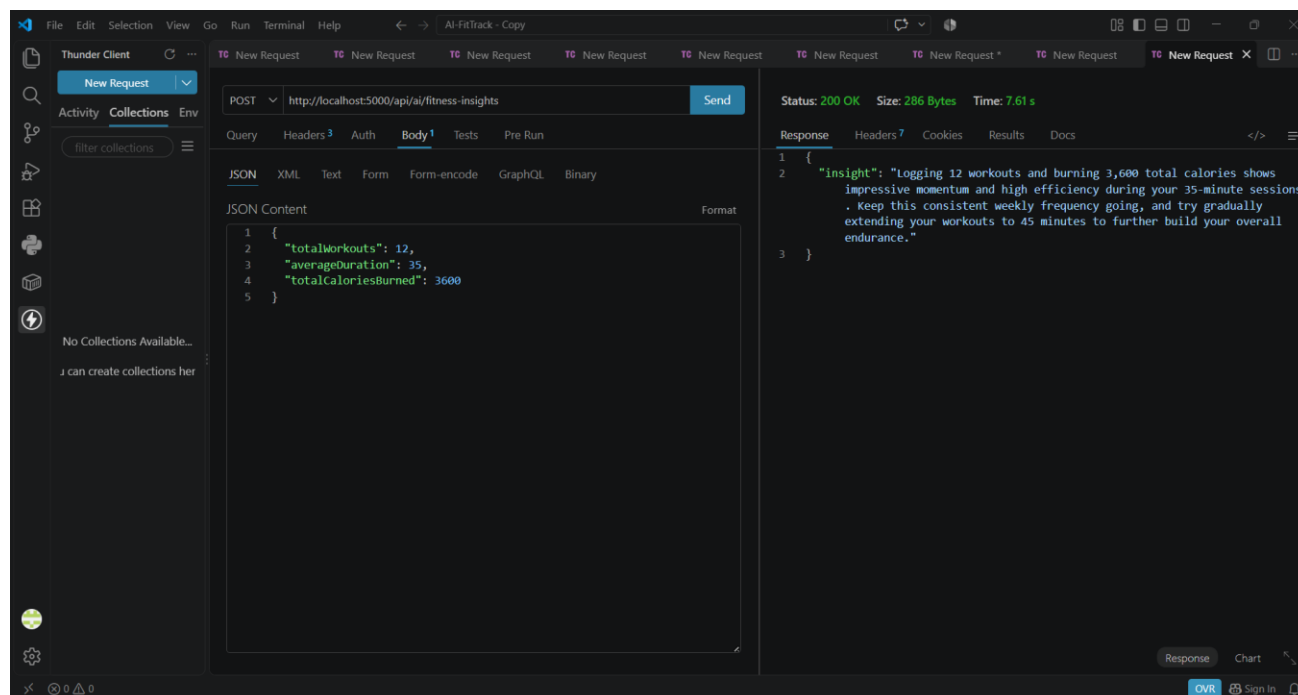
The screenshot shows the Thunder Client interface with a POST request to `http://localhost:5000/api/ai/workout-recommendation`. The request body is a JSON object:

```
1 {
2   "age": 24,
3   "fitnessGoal": "Weight loss",
4   "experience": "Beginner"
5 }
```

The response is a 200 OK status with a size of 1.24 KB and a time of 9.26 s. The response body is a JSON object:

```
1 {
2   "recommendation": "Commit to a three-day full-body strength training routine on non-consecutive days, such as Monday, Wednesday, and Friday, paired with two dedicated cardio days on Tuesday and Thursday. Begin every workout session with five minutes of light dynamic movement, including arm circles, bodyweight torso twists, and high knees, to prepare your joints and elevate your heart rate safely.\n\nFor the strength training sessions, perform three sets of ten to twelve repetitions for four fundamental exercises: bodyweight squats, incline push-ups off a sturdy bench, dumbbell Romanian deadlifts, and standing resistance band rows. Rest for ninety seconds between each set, prioritizing smooth and controlled form over heavy resistance. On your cardio days, complete thirty minutes of moderate-intensity brisk walking or stationary cycling, maintaining a pace where you can talk but not easily sing. \n\nExecute this exact routine for four weeks to build consistency before adding extra weight or time. Support your workouts by drinking plenty of water, aiming for seven to eight hours of sleep per night, and maintaining a slight daily calorie deficit with high protein intake to drive sustainable weight loss. Use the weekend for total rest and physical recovery."
3 }
```

Fitness Insights



The screenshot shows the Thunder Client interface with a POST request to `http://localhost:5000/api/ai/fitness-insights`. The request body is a JSON object:

```
1 {
2   "totalWorkouts": 12,
3   "averageDuration": 35,
4   "totalCaloriesBurned": 3600
5 }
```

The response is a 200 OK status with a size of 286 Bytes and a time of 7.61 s. The response body is a JSON object:

```
1 {
2   "insight": "Logging 12 workouts and burning 3,600 total calories shows impressive momentum and high efficiency during your 35-minute sessions. Keep this consistent weekly frequency going, and try gradually extending your workouts to 45 minutes to further build your overall endurance."
3 }
```

4.5 Error Handling and Security Testing

- Missing Authorization header is rejected with an authentication error.
- Invalid or failed JWT verification is rejected.
- Registration validates required user fields.
- Login validates credentials before issuing a token.
- Workout records are scoped to the authenticated user.
- Unknown routes are forwarded to centralized error handling.
- MongoDB connection failures cause the application startup process to terminate rather than continuing with an unavailable database connection.

The supplied architecture identifies JWT verification, user lookup and password exclusion as responsibilities of the authentication middleware. The User model handles bcrypt password hashing and comparison. The Workout controller also checks record ownership before individual retrieval and deletion.

4.6 Implementation Summary

The implementation demonstrates the major backend functions defined to the FitTrack project: server configuration, MongoDB connection, authentication, workout CRUD/search, AI integration, middleware and centralized error handling. The project remains within the assigned backend scope rather than adding unrelated frontend or administrative functionality.

The Skill Wallet screenshots supplied by the user show five epics and sixteen tasks, with Project Execution containing Steps for Project Execution, Output Screenshots and Demo Video. The GitHub repository should therefore preserve a direct relationship between each task and the evidence stored in the repository.

5. FUTURE ENHANCEMENT

5.1 Frontend Client

A dedicated React or mobile client could consume the existing REST API and provide a graphical interface for registration, workout management and AI results. The current project is intentionally documented as a backend API, so such a client should be treated as future work rather than an existing feature.

A frontend could also provide dashboards, charts and form validation while continuing to use the current authentication and workout endpoints. Any future client should preserve the backend's authenticated ownership model.

5.2 Expanded Fitness Data

The current implementation uses User and Workout models. Future versions could introduce additional fitness entities if the project scope is formally expanded and approved such as goals, progress records or notifications. These should only be added after the data model and Skill Wallet scope are approved.

Expanding the database would require corresponding schema design, API routes, controllers, validation and testing. The current documentation intentionally does not claim those collections exist.

5.3 AI Service Improvements

The Gemini integration can be further standardized so that model configuration is consistently loaded from the environment for every AI operation. Prompt templates could also be versioned and tested to improve consistency of generated responses.

Future work could include structured AI output validation, response-length controls and stronger error handling for unavailable models or provider-side limits. Such improvements should be tested against the exact Gemini SDK version used by the project.

5.4 Security and API Hardening

Future hardening can include stricter request validation, explicit update-field whitelisting and additional rate limiting if these requirements are added to the project scope. The current implementation already provides JWT protection and bcrypt password hashing.

Security changes should be introduced carefully because they can affect existing API behavior. Each change should be accompanied by regression testing of authentication, workout ownership and AI routes.

5.5 Deployment and Monitoring

A future deployment could place the backend behind a managed hosting environment and connect it to a managed MongoDB service. Monitoring could be added for server availability, API errors and provider failures.

Deployment should not be claimed in the current report because the verified project evidence concerns local execution with MongoDB Atlas rather than a production deployment.

6. CONCLUSION

6.1 Project Outcome

AI FitTrack demonstrates a modular backend approach to workout tracking and AI-assisted fitness guidance. The verified implementation provides secure authentication, user-specific workout management, search operations, MongoDB persistence and Gemini service integration. The project structure separates routes, controllers, middleware, models, configuration and AI service responsibilities.

The execution work also verified the core server, MongoDB connection, authentication workflow and workout API lifecycle. The AI recommendation functionality was tested after updating the Gemini model configuration, and the endpoint was verified through API testing.

6.2 Final Observation

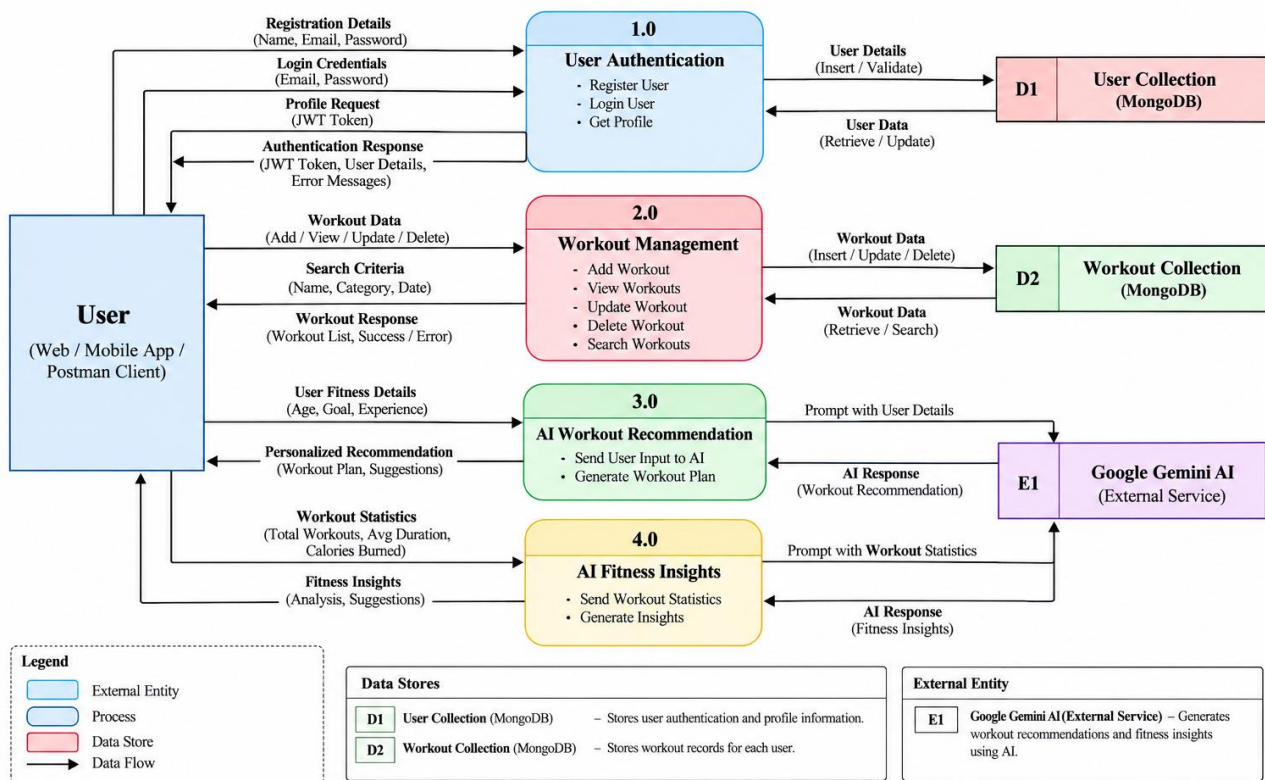
The project is suitable for GitHub-based Skill Wallet evidence when the repository is organized around the sixteen assigned tasks and the final screenshots and demo link are added. The documentation deliberately distinguishes implemented functionality from future enhancements and from features mentioned only in broader generic requirements.

The final repository should contain the source code, package metadata, environment template, documentation, diagrams and screenshots while excluding .env, node_modules, API keys, database passwords and JWT secrets. This keeps the repository both reviewable and appropriately compact.

APPENDIX A — DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) illustrates the overall flow of data between the user, application processes, database collections, and the Google Gemini AI service. The Level-1 DFD represents the major functional processes of AI FitTrack, including user authentication, workout management, AI workout recommendation, and AI fitness insights.

AI FITTRACK – LEVEL 1 DATA FLOW DIAGRAM



APPENDIX B — SAMPLE INPUT AND OUTPUT

Table B.1: Representative API Evidence

API	Sample Input	Expected Evidence
Register	name, email, password	201 Created + user/token response
Login	email, password	200 OK + JWT
Profile	Bearer JWT	200 OK + profile
Add Workout	workoutName, category, duration, caloriesBurned, workoutDate	201 Created
Search	q=Morning / Running / date	200 OK + matching records
AI Recommendation	age, fitnessGoal, experience	Final 200 OK evidence pending
AI Insights	totalWorkouts, averageDuration, totalCaloriesBurned	Final 200 OK evidence pending